Q1: What is the output?

```
int a = 10;
int *p = &a;
printf("%d", *p++);
```

Answer : 10

*Explanation: Post-increment operator ++ has higher precedence than dereference operator *. However, since it is post-increment, *p is evaluated first (which gives 10), and then the pointer p itself is incremented to point to the next int memory location.*

Q2: What is the output of the following code?

```
int arr[] = {10, 20, 30};
int *p = arr;
printf("%d", *++p);
```

Answer : 20

Explanation: Prefix ++ increments p first to point to arr[1] (value 20), and then dereferences it.

Q3: What does *p++ vs (*p)++ vs *++p vs ++*p do?

Answer : Detailed Behavior Summary

Explanation:

- *p++: Evaluates *p, then increments pointer p.
- (*p)++: Evaluates *p, then increments the value stored at p.
- *++p: Increments pointer p first, then dereferences it.
- ++*p: Increments the value pointed to by p first, then returns it

Q4: What is the result of adding 1 to a pointer?

Answer : 1 * sizeof(type_pointed_to)
Explanation: If int *p = 0x1000 (assuming 4-byte integers), p + 1 results in 0x1004.

Q5: Is pointer subtraction valid?

```
int arr[5] = {1, 2, 3, 4, 5};
int *p1 = &arr[1];
int *p2 = &arr[4];
printf("%td", p2 - p1);
```

Answer: 3
Explanation : Subtracting two pointers pointing to the same array yields the number of elements between them ((0x1010 - 0x1004) / sizeof(int)), not raw bytes.

Q6: Can you add two pointers?

Answer : No

Explanation: Adding two pointers (p1 + p2) is a compilation error because adding addresses produces an invalid memory reference.

Q7: What is a dangling pointer?

Answer : A pointer pointing to deallocated memory.

Explanation: A pointer pointing to a memory location that has been deallocated or freed (e.g., returning the address of a local variable from a function).

Q8: What is a wild pointer?

Answer : An uninitialized pointer.

Explanation: An uninitialized pointer that contains an arbitrary memory address. Reading or writing through it causes undefined behavior.

Q9: What is a NULL pointer vs a Void pointer?

Answer : NULL represents empty address; Void represents untyped address.

Explanation: NULL Pointer is initialized to 0 or NULL, indicating it points to nothing. Void Pointer (void *) is a generic

pointer type that can hold any data type address but cannot be dereferenced directly without casting.

Q10: What is the size of void * on a 64-bit architecture?

Answer : 8 bytes

Explanation: All pointers on a 64-bit architecture consume 8 bytes regardless of the type they point to.

Q11: What is the output?

```
int a = 0x12345678;
char *p = (char *)&a;
printf("0x%x", *p);
```

Answer : 0x78 (on Little-Endian systems)

Explanation: Little-Endian architecture stores the least significant byte (LSB) at the lowest memory address.

Q12: Is &p identical to p for a given pointer variable p?

Answer : No

Explanation: p is the address held inside the pointer variable.

&p is the memory address where the pointer variable itself resides.

Q13: What happens when you assign an integer directly to a pointer?

*int *p = (int *)0x1000;*

Answer : Assigns absolute address 0x1000 to p.

Explanation: Accessing *p will cause a Segmentation Fault unless 0x1000 is a valid, accessible mapped memory region.

Q14: What is ptrdiff_t and which standard header defines it?

Answer : A signed integer type.

Explanation: Defined in <stddef.h>, it is used to hold the result of subtracting two pointers.

Q15: What is the output?

```
int arr[3] = {1, 2, 3};
printf("%d", 0[arr]);
```

Answer : 1

Explanation: Array indexing a[i] is internally evaluated as *(a + i). By commutative law, *(0 + arr) is identical to arr[0].

Q16: What is const int *p vs int * const p?

Answer : Pointer to constant vs Constant pointer

Explanation: const int *p: Pointer to a constant integer (value cannot be modified, pointer can point elsewhere).

int * const p: Constant pointer to an integer (pointer address cannot change, value can be modified).

Q17: What does const int * const p mean?

Answer : Constant pointer to constant integer.

Explanation: Neither the pointer address nor the value stored at the memory location can be changed.

Q18: What is strict aliasing in C?

Answer : Compiler optimization rule.

Explanation: A C compiler rule assuming two pointers of different underlying types do not point to the same memory location (except char *), allowing aggressive optimization.

Q19: Why does casting int * to char * not violate strict aliasing?

Answer : Explicit exemption in C standard.
Explanation: The C standard explicitly allows character pointers (char *, unsigned char *) to alias any object type to permit low-level byte manipulation.

Q20: What is the output?

```
char *s = "Hello";
s[0] = 'h';
```

Answer : Segmentation Fault / Undefined Behavior
Explanation: "Hello" is a string literal stored in read-only memory (e.g., .rodata segment).

Q21: What is the difference between arr and & arr ?

```
int arr[5];
```

Answer : arr decays to int *; &arr is int (*)[5]
Explanation: arr decays to int * (points to arr[0]). &arr is a pointer to the entire array of type int (*)[5].

Q22: What is the output?

```
int arr[5] = {1, 2, 3, 4, 5};

printf("%p, %p\n", arr + 1, &arr + 1);
```

Answer : Addresses differ by array size.
Explanation: arr + 1 moves forward by sizeof(int) (4 bytes). &arr + 1 moves forward by sizeof(int[5]) (20 bytes).

Q23: What does int (*p)[5] mean?

Answer : Pointer to an array of 5 integers.
Explanation: p is a single pointer that points to an entire array of 5 integers.

Q24: What does int *p[5] mean?

Answer : Array of 5 integer pointers.
Explanation: p is an array containing 5 elements, where each element is a pointer to an int.

Q25: How do you declare a pointer to a function taking int and returning char?

Answer : char (*func_ptr)(int);
Explanation: Defines func_ptr as a function pointer accepting an integer argument and returning a char.

Q26: What is the output?

```
int arr[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

printf("%d", *(*(arr + 1) + 2));
```

Answer : ***6***
Explanation : **(arr + 1) yields pointer to row 1 ({4, 5, 6}). Adding 2 moves to element 2 of row 1 (6). Dereferencing gets value 6.*

Q27: How are 2D arrays laid out in memory in C?

Answer : Row-Major Order
Explanation: Elements are stored contiguously row by row in contiguous memory locations.

Q28: What is the output?

```
int a[5] = {10, 20, 30, 40, 50};
int *p = (int *)(&a + 1);
printf("%d", *(p - 1));
```

Answer : 50

Explanation: &a + 1 points immediately past the array. Casting to int * and subtracting 1 shifts the pointer back to a[4].

Q29: What happens when passing a 2D array to a function like void foo(int arr[][3])?

Answer : Column dimension must be specified.

Explanation: The compiler needs column size to perform pointer arithmetic (arr + i jumps by 3 * sizeof(int) bytes).

Q30: What is a Variable Length Array (VLA) pointer?

Answer : Pointer to array of runtime size.

Explanation: A pointer to an array whose runtime dimension is dynamically determined (introduced in C99).

Q31: Can sizeof evaluate pointer arithmetic at runtime?

```
int arr[5] = {1, 2, 3, 4, 5};
int *p = arr;
printf("%zu", sizeof(*p++));
```

Answer : 4 (or sizeof(int))
Explanation: p is NOT actually incremented at runtime because sizeof evaluates type information at compile time (except for VLAs).

Q32: What is the output?

```
char arr[] = "Hello";
char *p = "Hello";
printf("%zu, %zu", sizeof(arr), sizeof(p));
```

Answer : 6, 8 (on 64-bit system)
Explanation: arr is a char array containing 'H','e','l','l','o','\0'. p is a pointer variable taking 8 bytes.

Q33: How do you dynamically allocate a 2D array in contiguous memory?

Answer : Allocates single block using linear indexing.
Explanation: Ensures optimal cache locality and continuous memory allocation.

Q34: What is the decay rule for arrays in C?

Answer : Arrays convert to pointer to first element.
Explanation: An array expression implicitly converts (decays) into a pointer to its first element, EXCEPT when used as operand of sizeof, unary &, or string literal initialization.

Q35: What is the output?

```
int a[3] = {1, 2, 3};
int *p = a;
printf("%d", p[1]);
```

Answer : 2
Explanation: p[1] translates directly to *(p + 1).

Q36: Is &arr[0] identical to arr?

Answer : Yes
Explanation: Both evaluate to a pointer to the first element (int *).

Q37: Why is sizeof(arr) / sizeof(arr[0]) invalid inside a function receiving an array parameter?

Answer : Array parameters decay to pointers.
Explanation: Function parameters defined as int arr[] decay to int *arr. sizeof(arr) will yield pointer size (e.g., 8 bytes), not total array size.

Q38: What does int (*p())[5] represent?

Answer : Function returning pointer to array of 5 ints.
Explanation: p is a function taking no arguments and returning a pointer to an array of 5 integers.

Q39: What does int (*x[10])(int) represent?

Answer : Array of 10 function pointers.
Explanation: An array of 10 pointers to functions that take an int parameter and return an int.

Q40: What is the output?

```
int a[2][2] = {1, 2, 3, 4};
printf("%d", **a);
```

Answer: 1

*Explanation: *a dereferences 2D array to a pointer to first row. **a dereferences first row to first element a[0][0].*

Q41: What is Structure Padding?

Answer : Insertion of unused bytes for alignment.

Explanation: Extra unused bytes inserted by compiler between structure members to align data types with hardware memory boundary addresses.

Q42: What is Structure Packing?

Answer : Disabling padding bytes.

Explanation: Directing compiler to eliminate padding bytes using directives like #pragma pack(1) or __attribute__((packed)).

Q43: What is the size of this struct on a 32-bit machine?

```
struct Test {
    char a;
    int b;
    char c;
};
```

Answer : 12 bytes
Explanation: Layout: a (1 byte) + padding (3 bytes) + b (4 bytes) + c (1 byte) + trailing padding (3 bytes).

Q44: What is a Self-Referential Structure?

Answer : Struct containing pointer to its own type.
Explanation: Used extensively in linked data structures like linked lists, trees, and graphs.

Q45: Why cannot a structure contain a direct instance of itself?

Answer : Infinite recursive size.
Explanation: It would lead to infinite recursive sizing, making structure size undefined/infinite at compile time.

Q46: What is the output?

```
struct Data {
    int x;
    int y;
};
struct Data d = {10, 20};
int *p = (int *)&d;
```

```
printf("%d", *(p + 1));
```

Answer : 20
Explanation: Members x and y are laid out sequentially in memory. p + 1 points to y.

Q47: What is the offsetof() macro and where is it defined?

Answer : Returns byte offset of a structure member.
Explanation: Defined in <stddef.h>, it returns byte offset of a member within a structure.

Q48: How is offsetof() macro typically implemented under the hood ?

```
#define offsetof(type, member) ((size_t)&(((type *)0)->member))
```

Answer : Casts 0 to structure pointer.
Explanation: Evaluates address of member starting from dummy base address 0.

Q49: What is a Flexible Array Member (FAM) in C99?

```
struct Buffer {
   int len;
   char data[]; // FAM
};
```

Answer : Array without explicit size at end of struct.
Explanation: Used for allocating variable length structures dynamically in C99.

Q50: How do you dynamically allocate memory for a structure with a Flexible Array Member?

```
struct Buffer *b = malloc(sizeof(struct Buffer) + payload_size);
```

Answer : Allocates base structure plus extra buffer payload.
Explanation: Allows contiguous memory allocation for structure metadata and variable payload.

Q51: What is the output?

```
struct Point { int x, y; };
struct Point p1 = {1, 2};
```

```
struct Point *p = &p1;
printf("%d", p->x);
```

Answer : 1

Explanation : Arrow operator -> dereferences pointer and accesses member ((*p).x).

Q52: What are bit-fields in structures?

Answer : Specifying exact bit width for members.

Explanation: Used to compress memory layout by declaring specific bit widths.

Q53: Can you take the address of a bit-field member using &?

Answer : No

Explanation: Pointers operate on byte boundaries; bit-fields may start at arbitrary bit offsets. &flags.is_ready causes compilation error.

Q54: What is a memory alignment requirement?

Answer : Hardware restriction for variable placement.
Explanation: Hardware requirement that variables of size N bytes must reside at memory addresses divisible by N.

Q55: What happens on CPU level during misaligned memory access?

Answer : Performance penalty or Hardware Bus Error.
Explanation: On modern x86 CPU performance penalty occurs; on strict architectures like ARM/SPARC, raises hardware Bus Error exception.

Q56: How does member declaration order affect structure size?

Answer : Ordering largest to smallest minimizes padding.
Explanation: Placing largest members first reduces internal padding requirements.

Q57: What is the size of this struct on a 64-bit system?

```
struct A {
    char a;
```

```
    double b;
    int c;
};
```

Answer : ***24 bytes***

Explanation: *a (1 byte) + 7 pad + b (8 bytes) + c (4 bytes) + 4 pad = 24 bytes (8-byte alignment).*

Q58: How would you reorder members of struct A to minimize memory padding?

```
struct OptimizedA {
   double b; // 8 bytes
   int c;      // 4 bytes
   char a;   // 1 byte
};
```

Answer : **16 bytes**

Explanation: Total size reduces from 24 bytes to 16 bytes (13 bytes payload + 3 bytes padding).

Q59: Can you copy structure values using direct pointer dereference assignment *p2 = *p1?

Answer : Yes (*p2 = *p1)
Explanation: Performs a shallow bitwise copy of structure contents.

Q60: What dynamic memory overhead exists when allocating structures via malloc()?

Answer : 8-16 bytes allocation metadata.
Explanation: malloc header tracks allocation size and heap metadata immediately before returned pointer.

Q61: What is the purpose of the container_of macro in the Linux kernel?

Answer : Retrieves parent struct pointer from member pointer.
Explanation: Used extensively in Linux kernel to access outer structure from an embedded member pointer

Q62: Write the definition of the container_of macro.

Answer : Subtracts byte offset from member pointer.
Explanation: Calculates exact base pointer of parent structure

Q63: Can structures contain function pointers?

Answer : Yes

Explanation: Used extensively in C to implement Object-Oriented design patterns and Virtual Method Tables (VTABLEs).

Q64: What is the output?

```
struct Node { int val; };
struct Node n = {10};
void *v = &n;
printf("%d", ((struct Node*)v)->val);
```

Answer : 10

Explanation: Void pointer v cast back to struct Node* to access member.

Q65: What is a zero-length array extension in C?

Answer : char data[0];

Explanation: Non-standard GCC extension preceding C99 flexible array members char data[].

Q66: What is a Union in C?

Answer : Data type where members share memory location.

Explanation: Size equals size of largest member plus alignment tail padding.

Q67: What is the size of this Union?

```
union Data {
   int i;      // 4 bytes
   char c[9];  // 9 bytes
   double d;   // 8 bytes
    };
```

Answer : 16 bytes

Explanation: Largest field is c[9] (9 bytes). Alignment requires multiple of largest alignment requirement (double = 8 bytes), rounding 9 up to 16.

Q68: What is Type Punning?

Answer : Reading stored bytes as a different data type.

Explanation: Bypassing standard type system to read stored bytes of one data type as another data type.

Q69: Is Type Punning via unions explicitly valid in C99/C11 standards?

Answer : Yes in C99/C11.

Explanation: Explicitly permitted in C standard, though undefined/implementation-defined in C++.

Q70: What is the output?

```
union Convert {
    float f;
    unsigned int i;
};
union Convert c;
c.f = 1.0f;
printf("0x%X", c.i);
```

Answer : 0x3F800000

Explanation: *Represents IEEE 754 floating-point raw byte representation of 1.0f read as unsigned integer.*

Q71: How can unions be used to check machine endianness?

Answer : Reads low-order byte.
Explanation: If LSB 0x02 is stored at byte offset 0, system is Little-Endian.

Q72: What is a Tagged (Discriminated) Union?

Answer : Union paired with explicit type indicator enum.
Explanation: Safe pattern ensuring program tracks active stored member in union.

Q73: What is the structural difference in memory allocation between struct and union?

Answer : Sum vs Max member size.
Explanation: Struct size = sum of member sizes + padding. Union size = max member size + alignment padding.

Q74: What is the output?

```
union Test {
    int a;
    char b;
};
```

```
union Test t;
int *p1 = &t.a;
char *p2 = &t.b;
printf("%d", p1 == (int*)p2);
```

Answer : 1 (True)

Explanation: *All union fields begin at exact same base address (&t).*

Q75: Can unions contain structures as members?

Answer : Yes

Explanation: Widely used in networking stacks to parse varying packet headers sharing fixed payload regions.

Q76: What is an anonymous union in C?

Answer : Union without variable tag inside struct.

Explanation: Members are accessed directly (packet.code) without extra variable qualifier.

Q77: What happens when writing to one union member and reading from another?

Answer : Reinterprets underlying bit pattern.

Explanation: Reinterprets underlying memory bit pattern according to new type's representation.

Q78: Why is union type punning preferred over raw pointer casting?

Answer : Avoids strict-aliasing optimizer bugs.

Explanation: Union access explicitly communicates aliasing intentions to compiler, avoiding optimizations breaking code behavior.

Q79: What is the output?

```
union U {
    int arr[2];
    long long val;
};
union U u = {{10, 20}};
printf("%lld", u.val);
```

Answer : Implementation defined

Explanation: *Typically 0x000000140000000A on 64-bit Little Endian system representing packed array bytes.*

Q80: Can a union be passed by value to a function?

Answer : Yes

Explanation: Entire union memory block gets copied to function stack frame.

Q81: What is Memory-Mapped I/O (MMIO)?

Answer : Hardware registers mapped into CPU memory space.

Explanation: Hardware architectural technique mapping peripheral hardware registers directly to normal CPU memory address space.

Q82: Why is the volatile keyword required when mapping hardware registers to pointers?

Answer : Prevents compiler optimizations.

Explanation: Prevents compiler from optimizing away reads/writes, caching values in registers, or reordering accesses to hardware registers.

Q83: Declare a pointer to a volatile memory-mapped 32-bit hardware register at 0x40000000.

Answer : Maps volatile pointer to exact hardware address.
Explanation: Ensures every read/write hits hardware register directly.

Q84: What does volatile int * const p mean?

Answer : Constant pointer to volatile integer.
Explanation: Address cannot change; value at memory address can change asynchronously outside program control.

Q85: What does int * volatile p mean?

Answer : Volatile pointer to integer.
Explanation: Pointer address itself can change asynchronously (e.g., modified by signal handler or interrupt).

Q86: What is the output behavior?

```
int a = 10;
volatile int *p = &a;
int b = *p;
```

```
int c = *p;
```

Answer : ***Forces two distinct physical memory loads.***
Explanation: *Compiler generates two separate physical load instructions from memory address &a rather than reusing register contents.*

Q87: What is the purpose of the mmap() POSIX system call?

Answer : **POSIX memory mapping system call.**
Explanation: Maps files or physical device addresses directly into virtual memory address space of a process.

Q88: What is Internal vs External Memory Fragmentation?

Answer : **Internal vs External fragmentation.**
Explanation: Internal Fragmentation: Unused space inside allocated memory block (e.g. padding).
External Fragmentation: Total free memory is sufficient, but divided into non-contiguous blocks.

Q89: What is the sbrk() system call used for?

Answer : System call to alter process heap break address.
Explanation: Legacy Unix system call used to expand or shrink process data segment space dynamically.

Q90: What causes a Page Fault in OS memory management?

Answer : Interrupt raised by MMU.
Explanation: Interrupt raised by MMU (Memory Management Unit) when process accesses virtual memory page not currently mapped into physical RAM.

Q91: How does the OS detect NULL pointer dereferences at hardware level?

Answer : Accessing unmapped memory page 0x0.
Explanation: Operating system marks address 0x00000000 as unmapped/protected page. Accessing triggers hardware page fault resulting in Segmentation Fault.

Q92: What is a custom memory pool allocator?

Answer : Pre-allocating large memory block up front.
Explanation: Manually managing dynamic sub-allocations

using pointers to reduce malloc latency and eliminate fragmentation.

Q93: What exception applies to char * regarding strict aliasing rules?

Answer : **Pre-allocating large memory block up front.**
Explanation: Manually managing dynamic sub-allocations using pointers to reduce malloc latency and eliminate fragmentation.

Q94: Implement a C function signature for inspecting memory byte-by-byte.

```
void dump_bytes(const void *ptr, size_t size) {
   const unsigned char *p = (const unsigned char *)ptr;
   for (size_t i = 0; i < size; i++) {
      printf("%02X ", p[i]);
       }
   printf("\n");
}
```

Answer : **Iterates byte by byte through generic memory buffer.**
Explanation: Prints raw byte representation in hexadecimal format.

Q95: How do pointer out-of-bounds writes lead to Stack Overflow?

Answer : **Writing past local array memory bounds.**
Explanation: Writing past local array memory bound into adjacent stack frames, corrupting return frame address pointers (Stack Smashing).

Q96: What is the purpose of C99 restrict pointer qualifier?

Answer : **Tells compiler pointer is sole reference mechanism.**
Explanation: Pointer hint informing compiler that pointer is sole reference mechanism for accessing target memory location within scope, allowing aggressive loop optimizations.

Q97: What happens if restrict promise is violated?

```
void add(int * restrict a, int * restrict b, int *val) {
    *a += *val;
    *b += *val;
```

}

Answer : ***Undefined behavior***

Explanation: *Undefined behavior if caller passes overlapping pointer arguments (e.g. add(&x, &x, &x)).*

Q98: What are the C11 alignment macros (_Alignas, _Alignof)?

Answer : **alignas and alignof specifiers.**

Explanation: Allows explicit memory alignment parameters specification.

Q99: What is Pointer Provenance in compiler optimizations?

Answer : **Tracking valid derivation source of pointers.**

Explanation: Compiler analysis concept tracking valid derivation source of pointers. Creating pointers via invalid integer arithmetic can lead to compiler optimizations invalidating execution even if addresses match physically.

Q100: What is a Memory Leak and how do pointers relate to it?

Answer : **Unfreed dynamic memory.**

Explanation: Occurs when dynamically allocated heap memory (malloc) loses all pointer references without explicit free(). Prevent by establishing clear ownership models, zeroing pointers after freeing (p = NULL), and using tools like Valgrind or ASan.